

Cours sur le JavaScript

La représentation de la structure d'une page web offerte par un navigateur et exploitable via JavaScript est appelée **DOM**, pour **Document Object Model**.

Le DOM définit la structure d'une page et le moyen d'interagir avec elle : il s'agit d'une interface de programmation, ou API (*Application Programming Interface*).

Le site suivant référence le DOM

<https://www.w3.org/>

Le navigateur web permet de visualiser la hiérarchie des éléments d'une page web. Pour les visualiser dans Firefox, affichez les outils de développement en sélectionnant **Développement Web** puis **Outils de développement** dans le menu **Outils**. Cliquez ensuite sur l'onglet **Inspecteur** : la structure de la page s'affiche sous forme arborescente. Vous pouvez naviguer dans cette structure en "dépliant" les nœuds correspondant aux différentes balises HTML.

Ajouter une balise<script> pour exécuter le fichier JavaScript cours.js par exemple

```
<script src="../js/cours.js"></script>
</body>
```

Syntaxe

La variable **document** correspond à l'élément<html>.

On peut écrire avec les points

- document.body
- document.head

Affiche le contenu de l'enfant « 0 »

```
console.log(document.body.childNodes[0]);
```

Les espaces entre les balises ainsi que les retours à la ligne dans le code HTML sont considérés par le navigateur comme des nœuds textuels.

Affiche le contenu le nom du parent

```
var h1 = document.body.childNodes[1];  
console.log(h1.parentNode);
```

On peut parcourir tous les nœuds d'une page :

```
// Affiche les noeuds enfant du noeud body  
for (var i = 0; i < document.body.childNodes.length; i++)  
{  
    console.log(document.body.childNodes[i]);  
}
```

On peut aussi appeler l'enfant d'un enfant

```
console.log(document.body.childNodes[5].childNodes[1]);
```

Mais il y a mieux, tous les éléments du DOM possèdent une méthode **getElementsByTagName** (sous forme de tableau)

```
var titresElts = document.getElementsByTagName("h2");  
// Tous les titres h2  
console.log(titresElts[0]); // Affiche le premier  
titre h2  
console.log(titresElts.length); // Affiche 3
```

Les éléments du DOM disposent également d'une méthode **getElementsByClassName**.

```
// Tous les éléments ayant la classe "merveilles"  
var merveillesElts = document.getElementsByClassName("merveilles");  
for (var i = 0; i < merveillesElts.length; i++)
```

```
{  
    console.log(merveillesElts[i]);  
}
```

merveillesElts.length correspond aux nombres de classes « merveilles »

Enfin, chaque élément du DOM propose une méthode `getElementById`

Pour faciliter la sélection d'éléments suivant des critères complexes, le DOM s'est enrichi de deux nouvelles méthodes.

La première, nommée `querySelectorAll`, permet de rechercher des éléments à partir d'un **sélecteur CSS**

```
var couleur = document.querySelectorAll("div");
```

« couleur » devient un tableau de valeur

```
// Tous les paragraphes
```

```
console.log(document.querySelectorAll("p").length); // Affiche 3
```

```
// Tous les paragraphes à l'intérieur de l'élément identifié par "contenu"
```

```
console.log(document.querySelectorAll("#contenu p").length); // Affiche 2
```

```
// Tous les éléments ayant la classe "existe"
```

```
console.log(document.querySelectorAll(".existe").length); // Affiche 8
```

L'autre méthode de recherche d'éléments à partir d'un sélecteur CSS

s'appelle `querySelector`. Elle fonctionne de manière identique

à `querySelectorAll`, mais renvoie uniquement le premier élément correspondant au sélecteur passé en paramètre.

La propriété `innerHTML` permet de récupérer tout le contenu HTML d'un élément du DOM.

```
console.log(document.getElementById("contenu").innerHTML);
```

La propriété `textContent` renvoie tout le contenu textuel d'un élément du DOM, sans l'éventuel balisage HTML.

```
console.log(document.getElementById("contenu").textContent);
```

La méthode **getAttribute** peut être appliquée à un élément du DOM et renvoie la valeur de l'attribut passé en paramètre.

```
// L'attribut href du premier lien  
console.log(document.querySelector("a").getAttribute("href"));
```

Certains attributs sont directement accessibles sous la forme de propriétés. C'est notamment le cas pour les attributs id, href et value.

```
console.log(document.querySelector("a").href);
```

On peut vérifier la présence d'un attribut sur un élément grâce à la méthode **hasAttribute**, comme dans l'exemple ci-après.

```
if (document.querySelector("a").hasAttribute("target"))  
{  
    console.log("Le premier lien possède l'attribut target");  
}  
else  
{  
    console.log("Le premier lien ne possède pas l'attribut target");  
}
```

Dans une page web, une balise peut posséder plusieurs classes. La propriété **classList** permet de récupérer la liste des classes d'un élément du DOM. Elle s'utilise comme un tableau.

```
// Liste des classes de l'élément identifié par "antiques"  
var classes = document.getElementById("antiques").classList;  
console.log(classes.length); // Affiche 1 : l'élément possède une seule classe  
console.log(classes[0]); // Affiche "merveilles"
```

La propriété **innerHTML** peut être utilisée pour modifier le contenu HTML d'un élément du DOM.

```
// Modification du contenu HTML de la liste : ajout d'un langage
document.getElementById("langages").innerHTML += '<li id="c">C</li>';
```

innerHTML est souvent employée pour "vider" un élément de tout son contenu. Tapez l'exemple suivant pour le vérifier.

```
document.getElementById("langages").innerHTML = " ";
```

La propriété **textContent** permet de modifier le contenu textuel d'un élément du DOM.

```
// Modification du contenu textuel du premier titre
document.querySelector("h1").textContent += " de programmation";
```

La méthode **setAttribute** permet de définir la valeur de l'un des attributs d'un élément. Elle prend en paramètres le nom et la valeur de cet attribut.

```
// Définition de l'attribut "id" du premier titre
document.querySelector("h1").setAttribute("id", "titre");
```

on peut aussi écrire :

```
document.querySelector("h1").id = "titre";
```

On peut utiliser la propriété **classList** pour ajouter ou supprimer des classes à un élément du DOM.

```
var titreElt = document.querySelector("h1"); // Accès au premier titre h1
titreElt.classList.remove("debut"); // Suppression de la classe "debut"
titreElt.classList.add("titre"); // Ajout de la classe "titre"
console.log(titreElt);
```

La création d'un élément s'effectue grâce à une méthode nommée **createElement**

```
var pythonElt = document.createElement("li"); // Création d'un élément li
```

Il existe plusieurs techniques pour insérer un nouveau noeud dans le DOM. La plus courante consiste à appeler la méthode **appendChild**

```
var pythonElt=document.createElement("li");  
pythonElt.textContent="toto";  
document.getElementById("langages").appendChild(pythonElt); // Insertion du  
nouvel élément
```

Il est possible d'aboutir au même résultat en ajoutant au nouvel élément un noeud fils de type texte. Pour cela, on utilise la méthode **createTextNode**

```
var rubyElt = document.createElement("li"); // Création d'un élément li  
rubyElt.id = "ruby"; // Définition de son identifiant  
rubyElt.appendChild(document.createTextNode("Ruby")); // Définition de son  
contenu textuel  
document.getElementById("langages").appendChild(rubyElt); // Insertion du  
nouvel élément
```

On souhaite parfois insérer un nouvel élément ailleurs que comme dernier fils de son élément parent. Pour cela, il existe la méthode **insertBefore**.

Il existe également une méthode plus récente qui permet de définir encore plus précisément la position des éléments insérés : **insertAdjacentHTML**.

- **beforebegin**: avant l'élément existant lui-même.
- **afterbegin**: juste à l'intérieur de l'élément existant, avant son premier enfant.
- **beforeend**: juste à l'intérieur de l'élément existant, après son dernier enfant.
- **afterend**: après l'élément existant lui-même.

```
// Ajout d'un élément au tout début de la liste
```

```
document.getElementById('langages').insertAdjacentHTML("afterBegin",  
    '<li id="javascript">JavaScript</li>');
```

Le remplacement d'un élément du DOM par un autre s'effectue au moyen de la méthode **replaceChild**

```
var bashElt = document.createElement("li"); // Création d'un élément li  
bashElt.id = "bash"; // Définition de son identifiant  
bashElt.textContent = "Bash"; // Définition de son contenu textuel  
// Remplacement de l'élément identifié par "perl" par le nouvel élément  
document.getElementById("langages").replaceChild(bashElt,  
document.getElementById("perl"));
```

il est possible de supprimer un noeud à l'aide de la méthode **removeChild**

```
// Suppression de l'élément identifié par "bash"  
document.getElementById("langages").removeChild(document.getElementById(  
"bash"));
```

CSS, HTML et JavaScript

Les éléments du DOM disposent d'une propriété nommée **style** qui renvoie un objet représentant l'attribut `style` de l'élément. Les propriétés de cet objet correspondent aux propriétés CSS de l'élément. En définissant leur valeur depuis JavaScript, on modifie le style de l'élément.

```
var pElt = document.querySelector("p");  
pElt.style.color = "red";  
pElt.style.margin = "50px";
```

Certaines propriétés CSS s'écrivent sous la forme d'un nom composé. Pour utiliser ces propriétés en JavaScript, *il faut supprimer le tiret et écrire la première lettre du mot suivant en majuscule.*

L'exemple ci-dessous modifie les propriétés CSS `font-family` et `background-color` du paragraphe sélectionné plus haut

```
// ...
```

```
pElt.style.fontFamily = "Arial";
```

```
pElt.style.backgroundColor = "black";
```

Cette convention de nommage s'appelle la norme **camelCase**.

La bonne solution pour accéder au style d'un élément consiste à utiliser une fonction nommée **getComputedStyle**. Elle prend en paramètre un noeud du DOM et renvoie un objet représentant son style. On peut ensuite consulter les différentes propriétés CSS de cet objet.

```
var stylePara = getComputedStyle(document.getElementById("para"));

console.log(stylePara.fontStyle); // Affiche "italic"
console.log(stylePara.color); // Affiche la couleur bleue en
valeurs RGB
```

exemple n°1 :

Code HTML

```
1  <!doctype html>
2  <html>
3
4  <head>
5      <meta charset="utf-8">
6      <link rel="stylesheet" href="../../css/liensweb.css">
7      <title>Activité 1</title>
8  </head>
9
10 <body>
11
12     <h1>Activité 1</h1>
13     <!-- Les nouveaux éléments sont ajoutés dans cette balise -->
14
15     <div id="contenu">
16
17         <div class="lien">
18             <a href="https://openclassrooms.com" style="color:#428bca ; text-decoration:none" > So Foot</a>
19             <span> http://sofoot.com
20                 <br/> Ajouté par yann.usaille </span>
21         </div>
22
23         <div class="lien">
24             <a href="https://openclassrooms.com" style="color:#428bca ; text-decoration:none" > Guide
25             d'autodéfense numérique</a>
26             <span> http://guide.boum.org
27                 <br/> Ajouté par paulochon </span>
28         </div>
29
30         <div class="lien">
31             <a href="https://openclassrooms.com" style="color:#428bca ; text-decoration:none" > L'encyclopédie
32             en ligne Wikipedia</a>
33             <span> http://Wikipedia.org
34                 <br/> Ajouté par annie.zette </span>
35         </div>
36     <script src="../../js/liensweb.js"></script>
37 </body>
38
39 </html>
```

Et CSS

```
1  body {
2      font-family: "Helvetica Neue", Helvetica, Arial, sans-serif;
3      background-color: #eee;
4      margin-left: 30px;
5      margin-right: 30px;
6  }
7
8  span {
9      font-weight: normal;
10     font-size: 80%;
11 }
12
13 .lien {
14     background: white;
15     padding: 10px;
16     margin-bottom: 10px;
17 }
```

Equivalent en JavaScript

```

1 ▾ |var listeLiens = [
2 ▾   {
3       titre: "So Foot",
4       url: "http://sofoot.com",
5       auteur: "yann.usaille"
6   },
7 ▾   {
8       titre: "Guide d'autodéfense numérique",
9       url: "http://guide.boum.org",
10      auteur: "paulochon"
11  },
12 ▾   {
13       titre: "L'encyclopédie en ligne Wikipedia",
14       url: "http://Wikipedia.org",
15       auteur: "annie.zette"
16   }
17  ];
18
19
20 //déclaration des différentes variables sous forme de tableaux
21 //liés à "listeLiens" ce qui permet d'augmenter la liste à volonté
22 var liste=listeLiens.length;
23 var lienA=listeLiens.length;
24 var lienB=listeLiens.length;
25
26 //boucle des liens
27 for(var x=0; x<listeLiens.length; x++)
28 ▾ {
29     liste[x]=document.createElement("div"); //balise div
30     liste[x].setAttribute("class", "lien"); //associée à la classe lien du CSS
31
32     lienA[x]=document.createElement("a"); //balise a, fond blanc
33     lienA[x].href = listeLiens[x].url; //lien URL
34     lienA[x].style.textDecoration="none";
35     lienA[x].style.color="#428bca";
36     liste[x].appendChild(lienA[x]);
37
38     lienB[x]=document.createElement("span"); //utilisation du span
39     liste[x].appendChild(lienB[x]);
40
41     document.getElementById("contenu").appendChild(liste[x]);
42     lienA[x].appendChild(document.createTextNode(listeLiens[x].titre));
43     lienB[x].appendChild(document.createTextNode(" "+listeLiens[x].url));
44     lienB[x].appendChild(document.createElement("br"));
45     lienB[x].appendChild(document.createTextNode("Ajouté par "+listeLiens[x].auteur));
46 }
47

```

Événements

Référence des événements : <https://developer.mozilla.org/fr/docs/Web/Events>

Appelée sur un élément du DOM, la méthode **addEventListener** lui ajoute un gestionnaire pour un événement particulier. Cette méthode prend deux paramètres : le type de l'événement et la fonction qui gère l'événement. Cette fonction sera appelée à chaque fois que l'événement se déclenchera sur l'élément.

```
function clic()
{
    console.log("Clic !");
}

var boutonElt = document.getElementById("bouton");
// Ajout d'un gestionnaire pour l'événement click
boutonElt.addEventListener("click", clic);
```

autre méthode

```
var boutonElt = document.getElementById("bouton");
// Ajout d'un gestionnaire pour l'événement click
boutonElt.addEventListener("click",
    function ()
    {
        console.log("clic");
    }
);
```

Dans ce cas, la fonction n'est plus identifiée par un nom et ne peut plus être utilisée ailleurs dans le programme. Il s'agit d'une **fonction anonyme**. Les fonctions anonymes sont fréquemment utilisées en JavaScript.

Il peut arriver que vous ne souhaitiez plus gérer un type d'événement sur un élément du DOM. Dans ce cas, appelez la méthode **removeEventListener** sur cet élément, en lui passant en paramètre la fonction qui gérait l'événement.

La solution la plus courante pour réagir à l'appui sur une touche du clavier consiste à gérer les événements de type **keypress** déclenchés sur le corps de la page web (élément body du DOM, correspondant en JavaScript à la variable globale « document »).

La méthode **String.fromCharCode** permet de traduire cette valeur en une chaîne représentant le caractère.

Pour gérer l'appui et le relâchement sur n'importe quelle touche du clavier (pas seulement celles qui produisent des caractères), on utilise les événements **keydown** et **keyup**.

```
// Affiche des informations sur un événement clavier
```

```
function infosClavier(e)
{
    console.log("Evènement clavier : " + e.type + ",
    touche : " + e.keyCode);
}
```

```
function test(e)
{
    console.log("Vous avez appuyé sur la touche " +
    String.fromCharCode(e.charCode));
}
```

```
// Gestion de l'appui et du relâchement d'une touche
du clavier
```

```
document.addEventListener("keypress", test);
```

il est à noter ici que quand un événement, par exemple « keypress », est appelé la fonction qui lui est associée (test) renvoie les propriétés au travers de la variable « e » de chacune des fonctions.

```
document.addEventListener("keydown", infosClavier);
document.addEventListener("keyup", infosClavier);
```

Gestion de la souris

```
// Renvoie le nom du bouton souris à partir de son
code
```

```
function getBoutonSouris(code) {
    var bouton = "inconnu";
    switch (code) {
```

```

    case 0: // 0 est le code du bouton gauche
        bouton = "gauche";
        break;
    case 1: // 1 est le code du bouton du milieu
        bouton = "milieu";
        break;
    case 2: // 2 est le code du bouton droit
        bouton = "droit";
        break;
    }
    return bouton;
}

// Affiche des informations sur un événement souris

function infosSouris(toto)
{
    console.log("Evènement souris : " + toto.type +
    ", bouton " +
        getBoutonSouris(toto.button) + ", X : " +
    toto.clientX + ", Y : " + toto.clientY);
}

// Gestion du clic souris
document.addEventListener("click", infosSouris);

```

Fin du chargement de la page web

Il est possible de détecter la fin du chargement de la page en ajoutant un gestionnaire pour l'événement **load** sur l'objet **window**, qui représente la fenêtre du navigateur.

```

// Gestion de la fin du chargement de la page web
window.addEventListener("load", function ()
{
    console.log("Page entièrement chargée");

```

```
});
```

Fermeture de la page web

```
// Gestion de la fermeture de la page web

window.addEventListener("beforeunload", function (e) {

    var message = "On est bien ici !";
    e.returnValue = message; // Provoque une demande
    de confirmation (standard)

    return message; // Provoque une demande de
    confirmation (certains navigateurs)

});
```

Propagation d'un événement

Le DOM représente une page web sous la forme d'une hiérarchie de noeuds. Les événements déclenchés sur un noeud enfant vont se déclencher ensuite sur son noeud parent, puis sur le parent de celui-ci.

Si on prend l'exemple suivant :

Pour le HTML

```
<p id="para">Un paragraphe avec un
    <button id="propa">bouton</button> à l'intérieur
</p>
```

Pour le JavaScript

```
// Gestion du clic sur le document
document.addEventListener("click", function () {
    console.log("Gestionnaire document");
});

// Gestion du clic sur le paragraphe
document.getElementById("para").addEventListener("click", function () {
    console.log("Gestionnaire paragraphe");
});
```

```
});  
  
// Gestion du clic sur le bouton  
document.getElementById("propa").addEventListener("click", function (e) {  
    console.log("Gestionnaire bouton");  
});
```

Si on clique sur le bouton le résultat est le suivant

```
Gestionnaire bouton  
Gestionnaire paragraphe  
Gestionnaire document
```

En fait l'événement s'est propagé

Pour stopper cette propagation on utilise la fonction **stopPropagation**

```
// Gestion du clic sur le bouton  
document.getElementById("propa").addEventListener("click", function (e) {  
    console.log("Gestionnaire bouton");  
    e.stopPropagation(); // Arrêt de la propagation  
    de l'événement  
});
```

Modifier le comportement par défaut en cas d'événement

Il est possible d'annuler ce comportement par défaut en appelant la méthode **preventDefault** sur l'objet Event.

```
// Gestion du clic sur le lien interdit  
document.getElementById("interdit").addEventListener("click", function (e) {  
    console.log("Continuez plutôt à lire le cours  
;));  
    e.preventDefault(); // Annulation de la navigation  
    vers la cible du lien
```

```
});
```

Les formulaires

<http://www.startyourdev.com/html/tag-html-balise-input>

Une zone de texte permet à l'utilisateur de saisir du texte sur une ou plusieurs lignes. Une zone monoligne correspond à la balise HTML `<input type="text">`. Une zone multiligne est définie grâce à la balise `<textarea>`.

```
<label for="pseudo">Pseudo</label> :  
<input type="text" name="pseudo" id="pseudo" required>  
<span id="aidePseudo"></span>
```

Donne :

Pseudo :

Pour récupérer la valeur saisie on utilise la propriété `value`

```
var pseudoElt = document.getElementById("pseudo");  
pseudoElt.value = "MonPseudo";
```

Gestion du focus

Lorsqu'une zone de texte est la cible de saisie, on dit que cette zone possède le **focus**. L'utilisateur d'un formulaire qui clique sur une zone de saisie pour y taper une information déclenche l'apparition d'un événement de type `focus`. Inversement, le changement de cible de saisie provoque l'apparition d'un événement de type `blur` sur l'ancienne zone qui avait le focus.

```
// Affichage d'un message contextuel pour la saisie du  
pseudo  
pseudoElt.addEventListener("focus", function () {  
    document.getElementById("aidePseudo").textContent  
= "Entrez votre pseudo";  
});  
// Suppression du message contextuel pour la saisie du  
pseudo  
pseudoElt.addEventListener("blur", function (e) {
```



```
document.getElementById("aidePseudo").textContent  
= "";  
});
```

Cases à cocher

On définit une case à cocher dans un formulaire avec la balise

`<input type="checkbox">`.

```
<input type="checkbox" name="confirmation"  
id="confirmation">  
<label for="confirmation">M'envoyer un courriel de  
confirmation</label>
```

Donne :

☐ M'envoyer un courriel de confirmation

L'exemple de code ci-dessous gère les événements **change** sur la case à cocher pour afficher dans la console le choix effectué lorsque l'utilisateur clique dans la case ou sur le libellé associé.

```
// Affichage de la demande de confirmation d'inscription  
document.getElementById("confirmation").addEventListener("change", function (e)  
{  
    console.log("Demande de confirmation : " +  
e.target.checked);  
});
```

Boutons radio

Un groupe de boutons radio permet à l'utilisateur de faire un seul choix parmi plusieurs possibilités. On crée ces boutons avec des balises **<input type="radio">** possédant le même attribut **name** et des attributs **value** différents.

```
<input type="radio" name="abonnement" id="abonewspromo"  
value="abonewspromo">
```

```

<label for="abonewspromo">M'abonner à la newsletter et aux
promotions</label>
<br>
<input type="radio" name="abonnement" id="abonews"
value="abonews">
<label for="abonews">M'abonner uniquement à la
newsletter</label>
<br>
<input type="radio" name="abonnement" id="aborien"
value="aborien" checked>
<label for="aborien">Ne pas m'abonner</label>
<br>

```

Donne :

- ☐ M'abonner à la newsletter et aux promotions
- ☐ M'abonner uniquement à la newsletter
- ☒ Ne pas m'abonner

Listes déroulantes

On construit une liste déroulante au moyen d'une balise **<select>** dans laquelle on ajoute une balise **<option>** par choix possible dans la liste.

```

<label for="nationalite">Nationalité :</label>
<select name="nationalite" id="nationalite">
  <option value="FR" selected>Française</option>
  <option value="BE">Belge</option>
  <option value="SUI">Suisse</option>
  <option value="XX">Autre</option>
</select>

```

Donne :

Nationalité :

Française	▼
Française	
Belge	
Suisse	
Autre	

L'exemple de code suivant exploite les événements de type **changedéclenchés** sur la liste déroulante pour afficher le nouveau choix (attribut **value** de la balise **<option>** associée au choix).

```
// Affichage du code de nationalité choisi
document.getElementById("nationalite").addEventListener("change", function (e) {
    console.log("Code de nationalité : " + e.target.value);
});
```

Accès aux champs du formulaire

La balise `<form>` définissant un formulaire correspond à un élément du DOM. Cet élément possède une propriété `elements` rassemblant les champs de saisie du formulaire. On peut l'utiliser pour accéder à un champ à partir de son nom (attribut `name`) ou à partir de son indice dans (ordre d'apparition dans le formulaire).

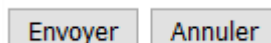
```
var form = document.querySelector("form");
console.log("Nombre de champs de saisie : " + form.elements.length); // Affiche 10
console.log(form.elements[0].name); // Affiche "pseudo"
console.log(form.elements.mdp.type); // Affiche "password"
```

Soumission du formulaire

Un formulaire est soumis lorsque l'utilisateur clique sur le bouton d'envoi. Ce bouton correspond à une balise `<input type="submit">`. Une balise `<input type="reset">` affiche un bouton qui réinitialise les données du formulaire.

```
<input type="submit" value="Envoyer">
<input type="reset" value="Annuler">
```

Donne :



```
var form = document.querySelector("form");

form.addEventListener("submit", function (e) {
    var pseudo = form.elements.pseudo.value;
    var mdp = form.elements.mdp.value;
    var courriel = form.elements.courriel.value;
```

```

        console.log("Vous avez choisi le pseudo " + pseudo + ", le
mot de passe " +
        mdp + " et le courriel " + courriel);
        if (form.elements.confirmation.checked)
        {
            console.log("Vous avez demandé une confirmation
d'inscription par courriel");
        } else
        {
            console.log("Vous n'avez pas demandé de confirmation
d'inscription par courriel");
        }

        switch (form.elements.abonnement.value) {
        case "abonewspromo":
            console.log("Vous êtes abonné(e) à la newsletter et aux
promotions");
            break;
        case "abonews":
            console.log("Vous êtes abonné(e) à la newsletter");
            break;
        case "aborien":
            console.log("Vous n'êtes abonné(e) à rien");
            break;
        default:
            console.log("Erreur : code d'abonnement non reconnu");
        }

        switch (form.elements.nationalite.value) {
        case "FR":
            console.log("Vous êtes français(e)");
            break;
        case "BE":
            console.log("Vous êtes belge");
            break;
        case "SUI":
            console.log("Vous êtes suisse");
            break;
        default:
            console.log("Erreur : code de nationalité non
reconnu");
        }
        e.preventDefault(); // Annulation de l'envoi des données
    });

```

Validation pendant la saisie

```

// Vérification de la longueur du mot de passe saisi

```

```

document.getElementById("mdp").addEventListener("input", function (e) {
    var mdp = e.target.value; // Valeur saisie dans le champ mdp
    var longueurMdp = "faible";
    var couleurMsg = "red"; // Longueur faible => couleur rouge
    if (mdp.length >= 8) {
        longueurMdp = "suffisante";
        couleurMsg = "green"; // Longueur suffisante => couleur verte
    } else if (mdp.length >= 4) {
        longueurMdp = "moyenne";
        couleurMsg = "orange"; // Longueur moyenne => couleur orange
    }
    var aideMdpElt = document.getElementById("aideMdp");
    aideMdpElt.textContent = "Longueur : " + longueurMdp; // Texte de l'aide
    aideMdpElt.style.color = couleurMsg; // Couleur du texte de l'aide
});

```

Validation à la fin de la saisie

On souhaite vérifier uniquement la présence du caractère @ dans le courriel saisi. Pour cela, on peut utiliser la méthode JavaScript [indexOf](#), qui permet de chercher une valeur dans une chaîne de caractères et renvoie la valeur -1 si cette valeur n'est pas trouvée.

```

// Contrôle du courriel en fin de saisie
document.getElementById("courriel").addEventListener("blur", function (e) {
    var valideCourriel = "";
    if (e.target.value.indexOf("@") === -1) {
        // Le courriel saisi ne contient pas le caractère @
        valideCourriel = "Adresse invalide";
    }
    document.getElementById("aideCourriel").textContent = valideCourriel;
});

```

Utilisation d'une expression régulière

Une **expression régulière**, également appelée expression rationnelle, définit un motif auquel on va comparer des chaînes de caractères pour trouver (ou non) des correspondances.

On définit une expression régulière JavaScript en plaçant son motif entre deux caractères `/`. La variable ainsi créée est un objet.

Le tableau suivant ne présente que quelques-unes des très nombreuses possibilités offertes pour définir le motif d'une expression régulière.

Motif	Correspondance si	test() === true	test() === false
<code>abc</code>	La chaîne contient "abc"	abc, abcdef, 123abc456	abdc, 1bca, adbc, ABC
<code>[abc]</code>	La chaîne contient soit "a", soit "b", soit "c"	abc, daef, bbb, 12c34	def, xyz, 123456, BBB
<code>[a-z]</code>	La chaîne contient n'importe quelle lettre minuscule de l'alphabet	abc, 12f43, _r_	123, ABC, _- _
<code>[0-9]</code> ou <code>\d</code>	La chaîne contient un chiffre	123, ab4c, a56	abc
<code>a.c</code>	La chaîne contient "a" suivi d'un caractère (n'importe lequel) suivi de "c"	abc, acc, 12a.c34	ac, abbc, ABC
<code>a\.c</code>	La chaîne contient "a.c"	a.c, a.cdef, 12a.c34	ac, abc
<code>a.+c</code>	La chaîne contient "a" suivi d'un ou plusieurs caractères (n'importe lesquels) suivi de "c"	abc, abbc, 12a\$ùc34	ac,bbc
<code>a.*c</code>	La chaîne contient "a" suivi de zéro ou plusieurs caractères (n'importe lesquels) suivi de "c"	abc, abbc, ac	ABC, bbc

Le site <https://regex101.com> permet de tester facilement ses expressions régulières. N'hésitez pas à l'utiliser pour mieux comprendre leur fonctionnement.

```
// Contrôle du courriel en fin de saisie
document.getElementById("courriel").addEventListener("
blur", function (e) {
    // Correspond à une chaîne de la forme xxx@yyy.zzz
    var regexCourriel = /.+@.+\.+\.+\/;
    var valideCourriel = "";
    if (!regexCourriel.test(e.target.value)) {
        valideCourriel = "Adresse invalide";
    }

    document.getElementById("aideCourriel").textContent =
valideCourriel;
});
```

Animation d'une page html

Utilisation de **setInterval** pour déclencher une action répétitive

```
setInterval(function, 1000); //en ms
```

Utilisation de **setTimeout** pour déclencher une action après un délai

```
setTimeout(function, 1000); //en ms
```

L'appel à **Number** permet de convertir le texte du compteur (chaîne de caractères) en valeur numérique.

Les valeurs des positions sont exprimées en pixels. Ce sont des chaînes de caractères de la forme "XXpx", ce qui nécessite l'utilisation de la fonction **parseFloat** pour les convertir en valeurs numériques avant d'effectuer des calculs.

La fonction **requestAnimationFrame** permet de demander au navigateur d'exécuter dès que possible une fonction qui met à jour une animation.

```
function animer() {
    // Code de l'animation
    // ...
    requestAnimationFrame(animer);
}
requestAnimationFrame(animer);
```

- Si l'animation n'est pas en temps réel et doit simplement se produire à intervalles réguliers, utilisez `setInterval`.
- Si l'animation est en temps réel et que vous savez qu'elle peut être effectuée via CSS, adoptez cette technique.
- Pour tous les autres cas, utilisez `requestAnimationFrame`.

EXEMPLE :

Test d'un mot de passe

HTML

```
<!doctype html>
<html>

<head>
  <meta charset="utf-8">
  <title>Vérification de mots de passe</title>
</head>

<body>
  <form>
    <p>
      <label for="mdp1">Saisissez le mot de passe</label> :
      <input type="password" name="mdp1" id="mdp1" required>
    </p>
    <p>
      <label for="mdp2">Confirmez le mot de passe</label> :
      <input type="password" name="mdp2" id="mdp2" required>
    </p>
    <input type="submit" value="Envoyer">
  </form>
  <p id="infoMdp"></p>
  <script src="../js/cours.js"></script>
</body>
</html>
```

JavaScript

```
var form = document.querySelector("form");

form.addEventListener("submit", function (e)
{
    var mdp_haut = form.elements.mdp1.value;
    var mdp_bas = form.elements.mdp2.value;
    var chiffre = /\d/; //test d'un chiffre dans le mdp
    if (mdp_haut!=mdp_bas)
    {
        document.getElementById("infoMdp").textContent
            ="Erreur: Les mots de passe ne sont pas identiques"
    }
    else if(!chiffre.test(form.elements.mdp1.value))
    {
        document.getElementById("infoMdp").textContent="Erreur:
        Le mot de passe ne contient pas de chiffre"
    }
    else if(mdp_haut.length<6)
    {
        document.getElementById("infoMdp").textContent="Erreur:
        Le mot de passe ne contient pas au moins 6 caractères"
    }
    else

        document.getElementById("infoMdp").textContent=
            "Mot de passe correct";
    e.preventDefault(); // Annulation de l'envoi des données
});
```

Affichage d'un item d'une liste déroulante

HTML

```
<!doctype html>
<html>

<head>
  <meta charset="utf-8">
  <!--link rel="stylesheet" href="../css/personnages.css"-->
  <title>Maisons et personnages</title>
</head>

<body>
  <h1>Quelques personnages de Game of Thrones</h1>
  <form>
    <label for="maison">Maison</label>
    <select name="maison" id="maison">
      <option value="" selected>Choisissez une maison</option>
    </select>
  </form>

  <p>
    <ul id="persos">
    </ul>
  </p>

  <script src="../js/cours.js"></script>
</body>
</html>
```

JavaScript

// Liste de quelques maisons de Game of Thrones. Chaque maison a un code et un nom

```
var form = document.querySelector("form");
```

```
var maisons = [  
  {  
    code: "ST",  
    nom: "Stark"  
  },  
  {  
    code: "LA",  
    nom: "Lannister"  
  },  
  {  
    code: "BA",  
    nom: "Baratheon"  
  },  
  {  
    code: "TA",  
    nom: "Targaryen"  
  }  
];
```

```
var option=[maisons.length];  
var liste=[];  
var num;
```

// Renvoie un tableau contenant quelques personnages d'une maison
function getPersonnages(codeMaison)

```
{  
  switch (codeMaison)  
  {  
    case "ST":  
      return ["Eddard", "Catelyn", "Robb", "Sansa", "Arya", "Jon Snow"];  
    case "LA":  
      return ["Tywin", "Cersei", "Jaime", "Tyrion"];  
    case "BA":  
      return ["Robert", "Stannis", "Renly"];  
    case "TA":  
      return ["Aerys", "Daenerys", "Viserys"];  
    default:  
      return [];  
  }  
}
```

//création de la liste déroulante

```
for(var i=0; i<maisons.length; i++)  
{  
  option[i]=document.createElement("option");  
  option[i].value=maisons[i].code;  
  option[i].textContent=maisons[i].nom;  
  document.getElementById('maison').appendChild(option[i]);  
}
```

```
//lors de l'appui sur un item du menu déroulant
document.getElementById("maison").addEventListener("change", function (e)
{
    //effacement
    //document.getElementById("persos").innerHTML=""; fontionne aussi
    if(document.getElementById("persos").childElementCount!=0)
    for(j=0; j<num; j++)
    {
        document.getElementById("persos").removeChild(liste[j]);
    }

    //affichage
    num=getPersonnages(e.target.value).length;
    for(var j=0; j<num; j++)
    {
        liste[j] = document.createElement("li");
        liste[j].textContent=getPersonnages(e.target.value)[j];
        document.getElementById("persos").appendChild(liste[j]);
    }

});
```